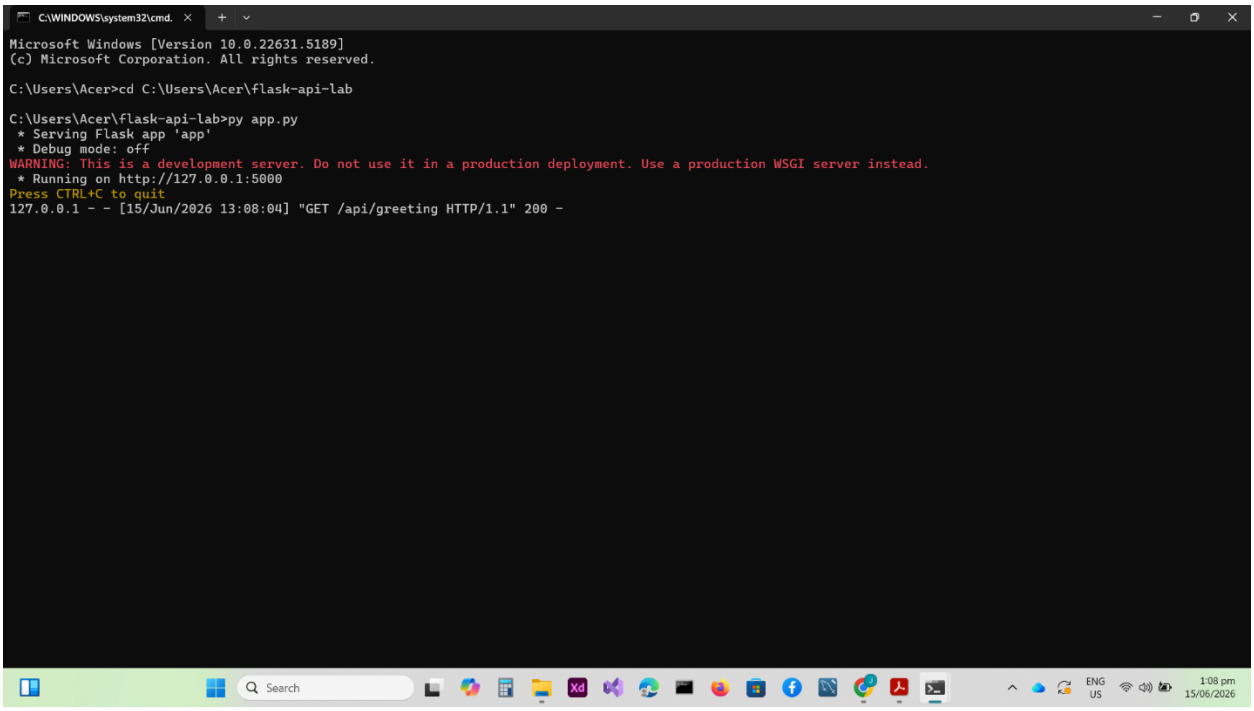
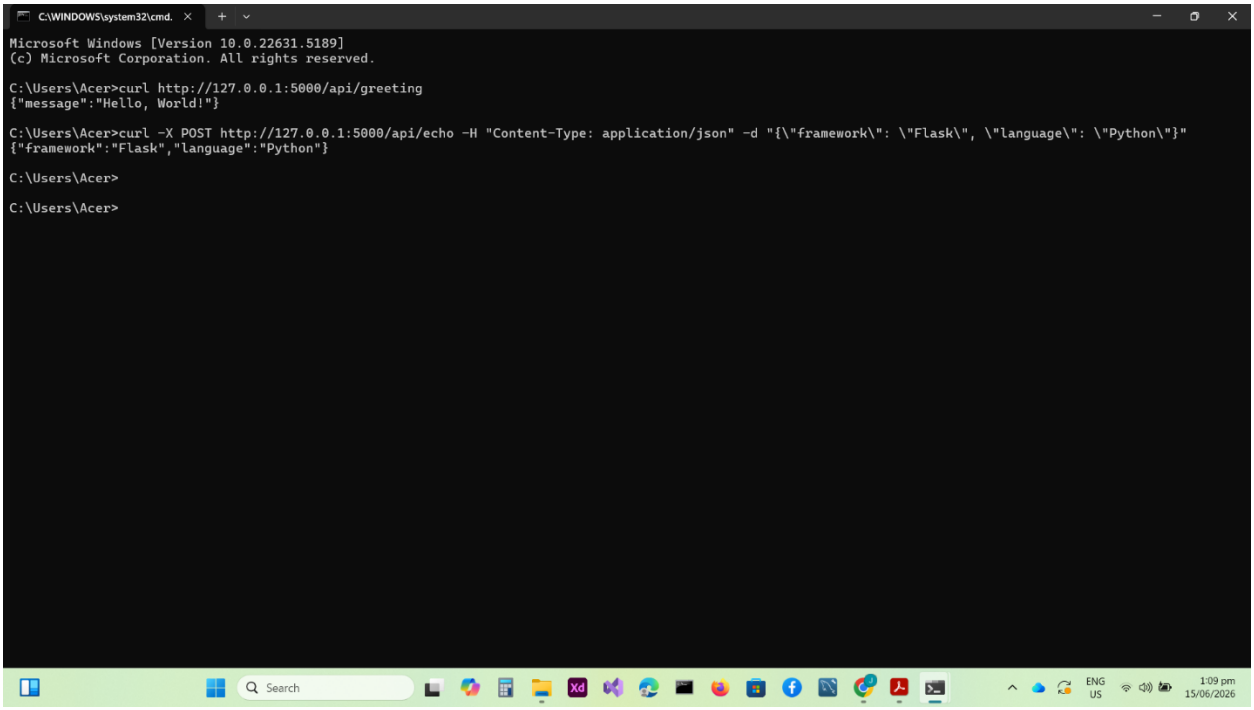

C:\WINDOWS\system32\cmd.
Microsoft Windows [Version 10.0.22631.5189]
(c) Microsoft Corporation. All rights reserved.
C:\Users\Acer>cd C:\Users\Acer\flask-api-lab
C:\Users\Acer\flask-api-lab>py app.py
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
127.0.0.1 - - [15/Jun/2026 13:08:04] "GET /api/greeting HTTP/1.1" 200 -
Search
ENG
US
1:08 pm
15/06/2026


C:\WINDOWS\system32\cmd.
Microsoft Windows [Version 10.0.22631.5189]
(c) Microsoft Corporation. All rights reserved.
C:\Users\Acer>curl http://127.0.0.1:5000/api/greeting
{"message":"Hello, World!"}
C:\Users\Acer>curl -X POST http://127.0.0.1:5000/api/echo -H "Content-Type: application/json" -d "{\"framework\": \"Flask\", \"language\": \"Python\"}"
{"framework":"Flask","language":"Python"}
C:\Users\Acer>
C:\Users\Acer>
Search
ENG
US
1:09 pm
15/06/2026

1. Syntactic Sugar: In Express, you used app.get(). In Flask, you use @app.route(..., methods=['GET']). Which do you find more intuitive, and why?

I find Express's app.get() and app.post() more intuitive because it's more direct, you immediately know the method just by reading the function name. Flask's @app.route() decorator requires you to read the methods=[...] parameter to know what HTTP method it handles, which adds a small extra step. However, Flask's approach is also flexible since one decorator can handle multiple methods at once.

2. The "Micro" Nature: Flask is called a "micro-framework." Based on your experience setting it up, how does it handle things like JSON parsing compared to Express?

Flask handles JSON parsing simply and cleanly using request.get_json() and jsonify(), but unlike Express, it doesn't include these features automatically, you have to import them manually from the flask module. Express with the express.json() middleware feels slightly more plug-and-play, while Flask gives you more control by keeping things minimal and only loading what you explicitly need.

3. Cross-Language Abstraction: Even though the language changed (JavaScript to Python), did the Endpoint, Method, and Payload stay the same? What does this tell you about the "Standardized Contract" of HTTP?

Yes, the endpoint path (/api/greeting, /api/echo), the HTTP methods (GET, POST), and the JSON payload format all stayed exactly the same even though we switched from JavaScript to Python. This shows that HTTP is a universal, language-agnostic contract. It doesn't matter what language or framework the server is built with — as long as it follows HTTP rules, any client can communicate with it the same way.